Sri Lanka Institute of Information Technology

IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Test Answer Submission

Student ID	IT24102822
Student Name	De Silva D. Y. S.
Campus/ Center name	SLIIT Malabe
Specialization	Data Science
Weekday/ Weekend	Weekend
Batch No	Y2.S2.WE.DS.0101

Question 01

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>


void PrintFibonacci(int n) {

   int a,b,next;

   a = 0;

   b = 1;

   printf("Fibonacci Series Up To %d terms: \n",n);


   for (int i=0; i<n; i++)  {

      printf("%d ",a);

      next = a+b;

      a = b;

      b = next;

   }

   printf("\n");

}


int main() {


   int n;

   pid_t pid;

   printf("Please enter a positive integer: ");

   scanf("%d",&n);
```

```
    if(n <= 0) {
        printf("Please enter a positive integer \n");
        return 1;
    }

    pid = fork();

    if(pid < 0) {
        printf("Fork Failed !\n");
        return 1;
    }

    else if(pid == 0) {
        printf("\n--- Child Process ---\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());
        PrintFibonacci(n);
    }

    else {
        wait(NULL);

        printf("\n--- Parent Process ---\n");
        printf("Child Process has Completed \n");
    }

    return 0;
}
```

```
  GNU nano 7.2                          Question_01.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void PrintFibonacci(int n) {
 int a,b,next;
 a = 0;
 b = 1;

 printf("Fibonacci Series Up To %d terms: \n",n);

 for (int i=0; i<n; i++)  {

        printf("%d ",a);
        next = a+b;
        a = b;
        b = next;
 }
 printf("\n");

}

int main() {

 int n;
 pid_t pid;
```

```
 printf("Please enter a positive integer: ");
 scanf("%d",&n);

 if(n <= 0) {
        printf("Please enter a positive integer \n");
        return 1;
 }

 pid = fork();

 if(pid < 0) {
        printf("Fork Failed !\n");
        return 1;
 }
 else if(pid == 0) {
        printf("\n--- Child Process ---\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());
        PrintFibonacci(n);
 }
 else {
        wait(NULL);
        printf("\n--- Parent Process ---\n");
        printf("Child Process has Completed \n");
 }
 return 0;
}
```

```
yashodara-de-silva@yashodara-de-silva-VirtualBox:~$ nano Question_01.c
yashodara-de-silva@yashodara-de-silva-VirtualBox:~$ gcc -oQuestion_01 Question_01.c
yashodara-de-silva@yashodara-de-silva-VirtualBox:~$ ./Question_01
Please enter a positive integer: 10

--- Child Process ---
Child PID: 5717
Child PPID: 5716
Fibonacci Series Up To 10 terms:
0 1 1 2 3 5 8 13 21 34

--- Parent Process ---
Child Process has Completed
```

Question 02

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <unistd.h>


int n;

int IsPrime(int num) {

if (num < 2)
     return 0;

   for (int i = 2; i * i <= num; i++) {

      if (num % i == 0)

         return 0;

   }

   return 1;

}

void* printPrimes(void* arg) {

   printf("\n Thread ID: %lu\n", pthread_self());

   printf("\n Process ID: %d\n", getpid());

   printf("Prime number up to %d are:\n", n);

   for (int i = 2;  i <= n ; i++) {

      if (IsPrime(i)) {

         printf("%d ", i);

      }

   }

   printf("\n");

   pthread_exit(NULL);

}
```

```
int main() {
    pthread_t tid;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Please Enter a positive Integer !\n");
        return 1;
    }

    if (pthread_create(&tid, NULL, printPrimes, NULL) != 0) {
        printf("Thread creation failed !\n");
        return 1;
    }

    pthread_join(tid, NULL);

    printf("\n Main Thread : Thread has finished execution \n");
 return 0;
}
```

```
GNU nano 7.2                    Question_02.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

int n;

int IsPrime(int num) {
 if (num<2)
        return 0;
 for (int i=2; i*i<=num; i++) {
        if(num % i == 0)
                return 0;
 }
return 1;
}

void*  printPrimes(void* arg) {
 printf("\n Tread ID : %lu\n",pthread_self());
 printf("\n Process ID : %d \n",getpid());
 printf("Prime number up to %d are: \n",n);
  for (int i=2; i<=n; i++) {
        if(IsPrime(i)) {
           printf("%d",i);
        }
  }
 printf("\n");
 pthread_exit(NULL);
```

```
int main() {
 pthread_t tid;

 printf("Enter a positive Integer : ");
 scanf("%d",&n);

 if(n<=0) {
        printf("Please Enter a positive Integer !\n");
        return 1;
 }

 if(pthread_create(&tid,NULL,printPrimes,NULL) != 0) {
        printf("Thread creation failed !\n");
        return 1;
 }

 pthread_join(tid,NULL);

 printf("\n Maint Thread : Thread has finished execution \n");

return 0;
}
```

```
yashodara-de-silva@yashodara-de-silva-VirtualBox:~$ nano Question_02.c
yashodara-de-silva@yashodara-de-silva-VirtualBox:~$ gcc -oQuestion_02 Question_02.c -lp
thread
yashodara-de-silva@yashodara-de-silva-VirtualBox:~$ ./Question_02
Enter a positive Integer : 10

 Tread ID : 135561130014400

 Process ID : 6050
Prime number up to 10 are:
2357

 Maint Thread : Thread has finished execution
```

Question 03

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

int sums[1000];

typedef struct {
   int i;
   int *arr;
   int start;
   int end;
} Data;

void* worker(void* arg) {
   Data* d=(Data*)arg;
   int sum = 0;
   int count = d->end - d->start + 1;

   for(int i = d->start; i <= d->end; i++) {
      sum += d->arr[i];
   }
   sums[d->i] = sum;

   printf("Thread ID: %lu\n",pthread_self());
   printf("Process ID: %d\n",getpid());
   printf("Start: %d \n End: %d\n", d->start, d->end);
   printf("Partial sum: %d\n\n", sum);

   return NULL;
}
```

```
int main() {
    int n,t;
    int total = 0;
    float avg;

    printf("Enter the size of array n: ");
    scanf("%d",&n);

    if(n <= 0) {
        printf("Array size must be positive !");
        return 1;
    }
    int arr[n];

    for(int i=0; i<n; i++) {
        printf("Enter value[%d]: ",i);
        scanf("%d",&arr[i]);
    }
    printf("Enter number of threads t (t <= n): ");
    scanf("%d",&t);

    if (t<=0 || t>n) {
        printf("Number of threads should be positive and less than or equal to array size !");
        return 1;
    }
    printf("\n");

    pthread_t tids[t];
    Data d[t];

    int sizept = n / t;
    int extra = n % t;
    int start = 0;
```

```
    for(int i=0; i<t; i++) {
        int end = start + sizept - 1;

        if (i < extra) {
            end = end +1;
        }

        d[i].i = i;
        d[i].arr = arr;
        d[i].start = start;
        d[i].end = end;

        if(pthread_create(&tids[i],NULL,worker,&d[i]) != 0) {
            printf("Thread creation failed !\n");
            return 1;
        }

        start = end + 1;
    }
    for(int i=0; i<t; i++) {
        pthread_join(tids[i],NULL);
    }

    for(int i=0; i<t; i++) {
        total += sums[i];
    }

    avg = (float)total / n;

    printf("Total Sum: %d\n",total);
    printf("Average: %.2f\n",avg);

    return 0;
}
```

```
  GNU nano 7.2                                          Question_03.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

int sums[1000];

typedef struct {
 int i;
 int *arr;
 int start;
 int end;
}Data;

void* worker(void* arg) {
 Data* d=(Data*)arg;
 int sum = 0;
 int count = d->end - d->start + 1;

 for(int i = d->start; i <= d->end; i++) {
        sum += d->arr[i];
 }
 sums[d->i] = sum;

 printf("Thread ID: %lu\n",pthread_self());
 printf("Process ID: %d\n",getpid());
 printf("Start: %d \n End: %d\n", d->start, d->end);
 printf("Partial sum: %d\n\n", sum);
```

```
 return NULL;
}

int main() {
 int n,t;
 int total = 0;
 float avg;

 printf("Enter the size of array n: ");
 scanf("%d",&n);

 if(n <= 0) {
        printf("Array size must be positive !");
        return 1;
 }

 int arr[n];

 for(int i=0; i<n; i++) {
        printf("Enter value[%d]: ",i);
        scanf("%d",&arr[i]);
 }

 printf("Enter number of threads t (t <= n): ");
 scanf("%d",&t);
```

```
GNU nano 7.2                                        Question_03.c
if (t<=0 || t>n) {
        printf("Number of threads should be positive and less than or equal to array size !");
        return 1;
}

printf("\n");

pthread_t tids[t];
Data d[t];

int sizept = n / t;
int extra = n % t;
int start = 0;

for(int i=0; i<t; i++) {
        int end = start + sizept - 1;

        if (i < extra) {
                end = end +1;
        }

        d[i].i = i;
        d[i].arr =  arr;
        d[i].start = start;
        d[i].end = end;
```

```
        if(pthread_create(&tids[i],NULL,worker,&d[i]) != 0) {
                printf("Thread creation failed !\n");
                return 1;
        }

        start = end + 1;
 }

 for(int i=0; i<t; i++) {
        pthread_join(tids[i],NULL);

 }

 for(int i=0; i<t; i++) {
        total += sums[i];
 }

 avg = (float)total / n;

 printf("Total Sum: %d\n",total);
 printf("Average: %.2f\n",avg);

 return 0;

}
```

```
yashodara-de-silva@yashodara-de-silva-VirtualBox:~$ nano Question_03.c
yashodara-de-silva@yashodara-de-silva-VirtualBox:~$ gcc -oQuestion_03 Question_03.c -lpthread
yashodara-de-silva@yashodara-de-silva-VirtualBox:~$ ./Question_03
Enter the size of array n: 5
Enter value[0]: 67
Enter value[1]: 90
Enter value[2]: 5
Enter value[3]: 100
Enter value[4]: 78
Enter number of threads t (t <= n): 2

Thread ID: 134591801190080
Thread ID: 134591809582784
Process ID: 6302
Start: 0
 End: 2
Partial sum: 162

Process ID: 6302
Start: 3
 End: 4
Partial sum: 178

Total Sum: 340
Average: 68.00
```